Since it's unclear whether these additional data points are needed for the long term, and since they're unlikely to ever be joined on, you decide to store this data as JSON compatible text in a denormalized document style representation. Another benefit of this approach is that you'll be able to append more attributes to this list over time. On the downside, you're wondering whether you'll be able to efficiently query this data or not when the row counts in this table grow.

To store JSON compatible strings you can choose between column types of json or jsonb. JSONB supports indexes on the data so you decide to use JSONB.

Additional details about the attributes are:

- bags_in_trunk (a number)
- music_on (true or false)
- water_offered (true or false)

In the snippet below, \gset is used to create a *dollar quoted literal* (See: *Defining multi-line strings in psql*[6]).

This is like a Heredoc or multi-line string in Ruby and is used here to better format the JSON string for printing purposes.

After making the variable json_string in psql, the variable is referenced in the statement SELECT jsonb_pretty(:json_string);.

Open psql and run this command:

```
SELECT
$$'{
  "ride_details": {
    "bags_in_trunk": 1,
    "music_on": true,
    "water_offered": true
  }
}'$$ AS json_string \gset
```

You've now set the json_string variable. Referencing :json_string inside the JSONB_PRETTY() function below will format it nicely. Besides formatting, this also ensures the JSON is valid.

Run this statement in psql:

```
SELECT jsonb_pretty(:json_string);

         jsonb_pretty
------------------------------
```

6. https://stackoverflow.com/a/31457183